Universite de Rouen Normandie

Master 1 – Genie Informatique et Logiciel

Projet Gestion de Flotte de vehicules

Rapport de Semaine 3

Microservice Véhicules

Etudiant :
Ahcene Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Annee universitaire :
2025 – 2026

Semaine :
3 / 7

28 mars 2026

Table des matières

1 Introduction et objectifs de la semaine **2**

2 Mise en œuvre du microservice de gestion des véhicules **3**
2.1 Modèle de données et Soft Delete . . . 3
2.2 Couche Accès aux Données (Repository) . . . 3
2.3 Logique Métier et DTOs . . . 4
2.4 Contrôleur REST . . . 4

3 Tests de l'API REST avec Postman **5**
3.1 CRUD Véhicules . . . 5
3.2 Gestion des assignations . . . 9
3.3 Bilan des tests . . . 10

4 Architecture événementielle avec Apache Kafka **11**
4.1 Événements produits . . . 11
4.2 Consommation et validation des messages . . . 11

5 Résolveurs GraphQL **12**
5.1 Architecture des résolveurs . . . 12
5.2 Résolveurs Véhicules . . . 13
5.3 Cohérence REST / GraphQL . . . 14
5.4 Gestion des stubs . . . 14

6 Sécurisation avec Keycloak et RBAC **15**
6.1 Configuration de Keycloak . . . 15
6.2 Intégration Spring Security . . . 15
6.2.1 Mapping des rôles . . . 15
6.3 Contrôle d'accès (RBAC) . . . 16

7 Instrumentation et Observabilité Distribuée **17**
7.1 Collecte des données via l'Agent Java . . . 17
7.2 Configuration de l'exportation (Docker Compose) . . . 17
7.3 Visualisation et Diagnostic dans Grafana . . . 17

8 Tests automatisés et couverture de code **19**
8.1 Stratégie de tests . . . 19
8.2 Mesure de la couverture avec JaCoCo . . . 19
8.3 Analyse des résultats . . . 19

9 Défis techniques et résolutions **21**
9.1 Incompatibilité de version Keycloak et initialisation . . . 21
9.2 Gestion des dépendances Kafka . . . 21
9.3 Isolation de l'environnement de test . . . 21

1. Introduction et objectifs de la semaine

Objectif de la semaine 3 : Développer le premier microservice fonctionnel du service véhicules en respectant les bonnes pratiques de qualité logicielle : tests, couverture de code, et observabilité intégrée dès le départ.

La semaine 3 du projet marque le passage de l'infrastructure au développement métier. L'équipe s'est concentrée sur la conception et l'implémentation du service de véhicules qui est le socle du système de gestion de flotte.

Les grandes étapes réalisées sont les suivantes :

1. Développement de l'**API REST** complète (CRUD véhicules avec validation des données)
2. Intégration d'**Apache Kafka** en tant que producer/consumer pour les événements métier
3. Création des **résolveurs GraphQL** pour l'agrégation future entre services
4. **Tests unitaires (>80% couverture) et d'intégration**
5. Instrumentation du service avec le **SDK OpenTelemetry** (traces, métriques, logs structurés)

2. Mise en œuvre du microservice de gestion des véhicules

L'une des briques centrales du système est le microservice de gestion de la flotte. Cette section détaille l'implémentation du CRUD (Create, Read, Update, Delete) pour l'entité `Vehicle`, en suivant une architecture en couches propre à Spring Boot.

2.1. Modèle de données et Soft Delete

L'entité `Vehicle` est conçue pour être persistée dans une base de données PostgreSQL. Une particularité de l'implémentation est l'usage du **Soft Delete** : au lieu de supprimer physiquement une ligne, nous renseignons un champ `deletedAt`.

```
@Entity
@Table(name = "vehicles")
public class Vehicle {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Column(name = "plate_number", nullable = false, unique = true)
    private String plateNumber;

    @Enumerated(EnumType.STRING)
    @JdbcTypeCode(SqlTypes.NAMED_ENUM)
    private VehicleStatus status = VehicleStatus.available;

    private OffsetDateTime deletedAt; // Pour le Soft Delete
}
```

Listing 1 – Entité Vehicle avec énumérations PostgreSQL

2.2. Couche Accès aux Données (Repository)

Pour supporter le mécanisme de suppression logique, le `VehicleRepository` définit des requêtes JPQL spécifiques qui filtrent les enregistrements dont la date de suppression est nulle.

```
@Repository
public interface VehicleRepository extends JpaRepository<Vehicle,
   UUID> {
    @Query("SELECT v FROM Vehicle v WHERE v.deletedAt IS NULL")
    List<Vehicle> findAllActive();

    @Query("SELECT v FROM Vehicle v WHERE v.id = :id AND v.
       deletedAt IS NULL")
    Optional<Vehicle> findByIdActive(UUID id);
}
```

Listing 2 – Requêtes personnalisées pour le Soft Delete

2.3. Logique Métier et DTOs

L'exposition des données ne se fait pas directement via l'entité, mais par des **Java Records** (DTOs), garantissant l'immutabilité et la validation des entrées.

— **Validation** : Utilisation de `@NotBlank` et `@Min` dans `VehicleInput` pour valider les payloads.

— **Mapping** : Le service transforme les entités en `VehicleResponse` avant de les retourner au contrôleur.

2.4. Contrôleur REST

Le `VehicleController` expose les points de terminaison nécessaires à la gestion de la flotte. Voici le récapitulatif des routes implémentées :

Méthode	Point de terminaison	Action
GET	`/vehicles`	Liste tous les véhicules actifs
GET	`/vehicles/{id}`	Détails d'un véhicule spécifique
POST	`/vehicles`	Création d'un véhicule (Validation active)
PUT	`/vehicles/{id}`	Mise à jour des informations
PATCH	`/vehicles/{id}/status`	Modification rapide du statut
DELETE	`/vehicles/{id}`	Suppression logique (Soft Delete)

```
@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteVehicle(@PathVariable UUID id) {
    service.deleteVehicle(id);
}
```

Listing 3 – Extrait du contrôleur pour la suppression logique

Cette structure assure que les données critiques restent en base pour la traçabilité (historique des livraisons) tout en étant invisibles pour les opérations courantes de l'application.

3. Tests de l'API REST avec Postman

> **Objectif :** Valider le bon fonctionnement de l'ensemble des endpoints du service Véhicules via des tests manuels réalisés avec **Postman**, couvrant les opérations CRUD ainsi que la gestion des assignations.

L'ensemble des routes exposées par le service Véhicules ont été testées manuellement à l'aide de Postman. Les tests couvrent les deux ressources principales : les **véhicules** et leurs **assignations**. Chaque requête a retourné le code HTTP attendu, confirmant la conformité de l'API aux standards REST.

3.1. CRUD Véhicules

POST /vehicles — Ajout d'un véhicule

La création d'un véhicule s'effectue via une requête `POST` sur `/vehicles`. Le corps de la requête contient les informations du véhicule (`plateNumber`, `brand`, `model`, `fuelType`, `mileageKm`, `vin`, `payloadCapacityKg`, `cargoVolumeM3`). Le service retourne un **201 Created** avec l'objet créé, incluant l'UUID généré automatiquement et le statut par défaut `available`.

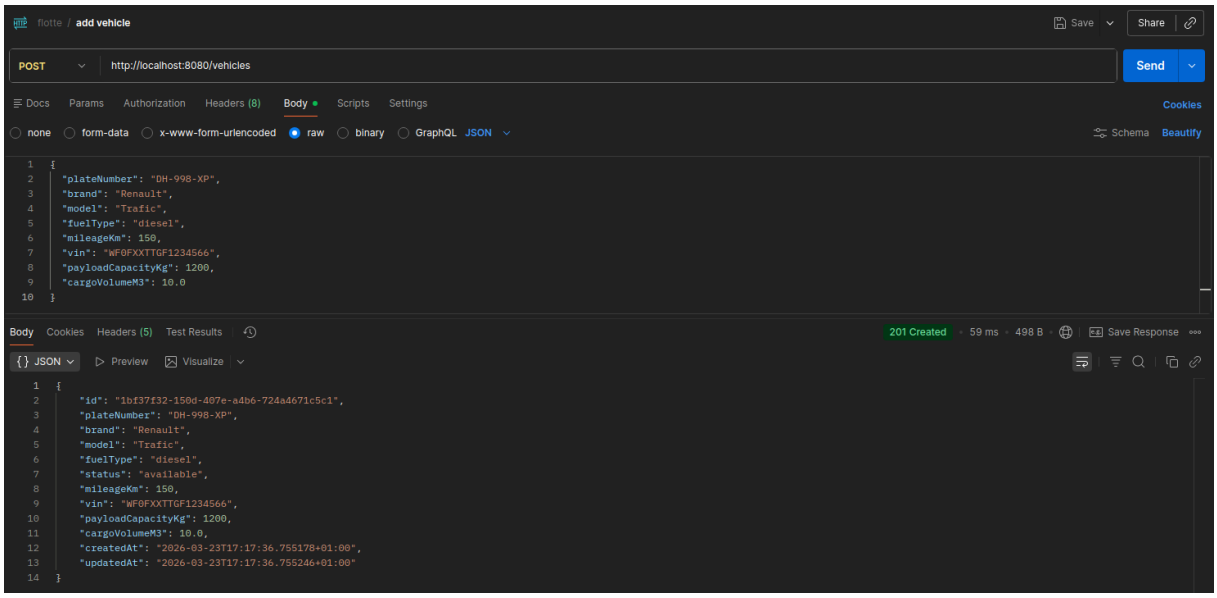

FIGURE 1 – POST /vehicles — Création d'un véhicule (201 Created)

	id [PK] uuid	plate_number character varying (20)	brand character varying (100)	model character varying (100)	fuel_type fuel_type	mileage_km integer	status vehicle_status
1	1bf37f32-150d-407e-a4b6-724a4671c5c1	DH-998-XP	Renault	Trafic	diesel	150	on_delivery
2	277223db-85d2-4543-8542-9c179ed393...	DH-999-XP	Ford	Transit	diesel	200	in_maintenance

FIGURE 2 – POST /vehicles — Détails

GET /vehicles — Liste de tous les véhicules

Une requête `GET` sur `/vehicles` retourne la liste des véhicules non supprimés. Le mécanisme de **soft delete** est ici mis en évidence : seuls les véhicules actifs sont retournés, même si la base de données en contient davantage. Le serveur répond avec un **200 OK**.

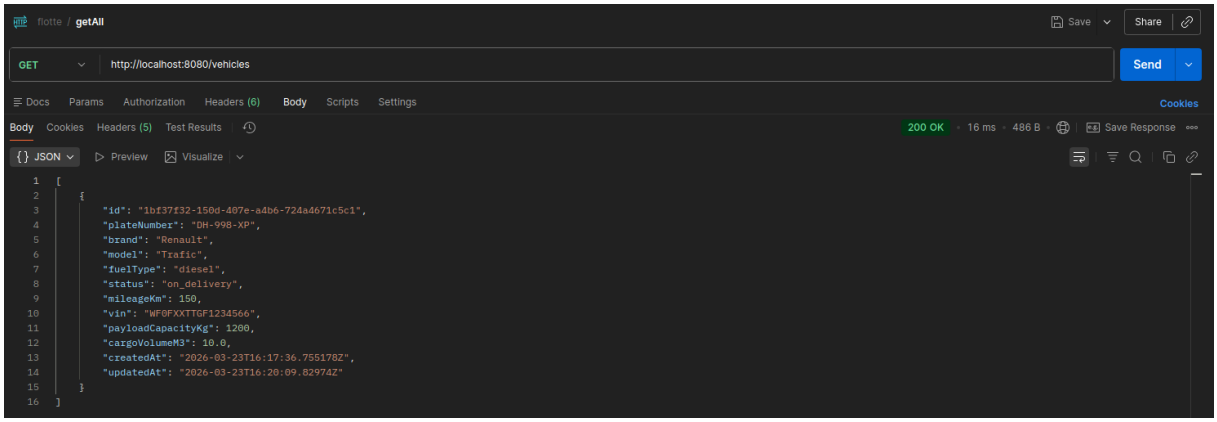

FIGURE 3 – GET /vehicles — Récupération de la liste des véhicules (200 OK)

GET /vehicles/ :id — Récupération par identifiant

La récupération d'un véhicule précis s'effectue en passant son UUID en paramètre de chemin. Le service retourne un **200 OK** avec l'intégralité des champs du véhicule correspondant.

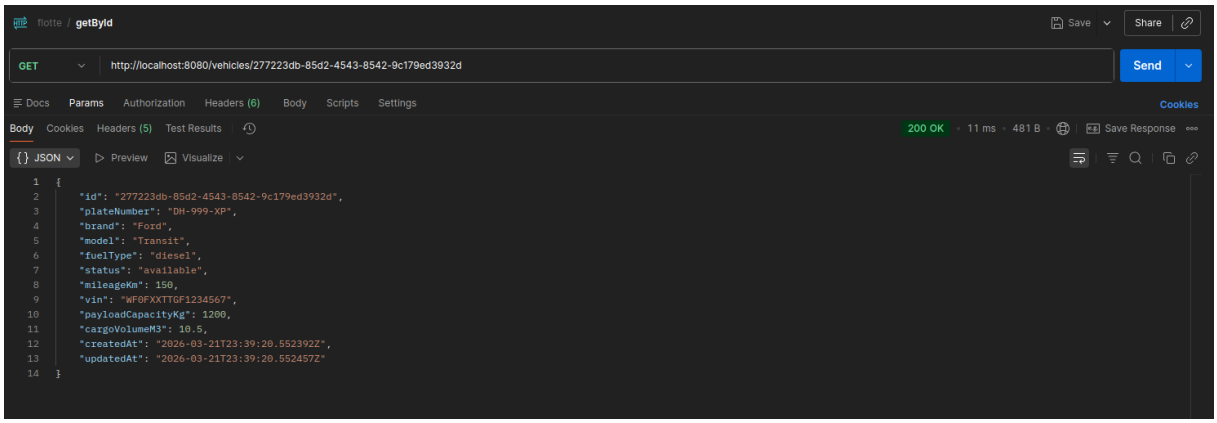

FIGURE 4 – GET /vehicles/ :id — Récupération d'un véhicule par UUID (200 OK)

PUT /vehicles/ :id — Mise à jour complète

La mise à jour d'un véhicule via `PUT` remplace les champs modifiables (`mileageKm`, `brand`, `model`, etc.). Le service retourne un **200 OK** avec les données mises à jour et un `updatedAt` rafraîchi.

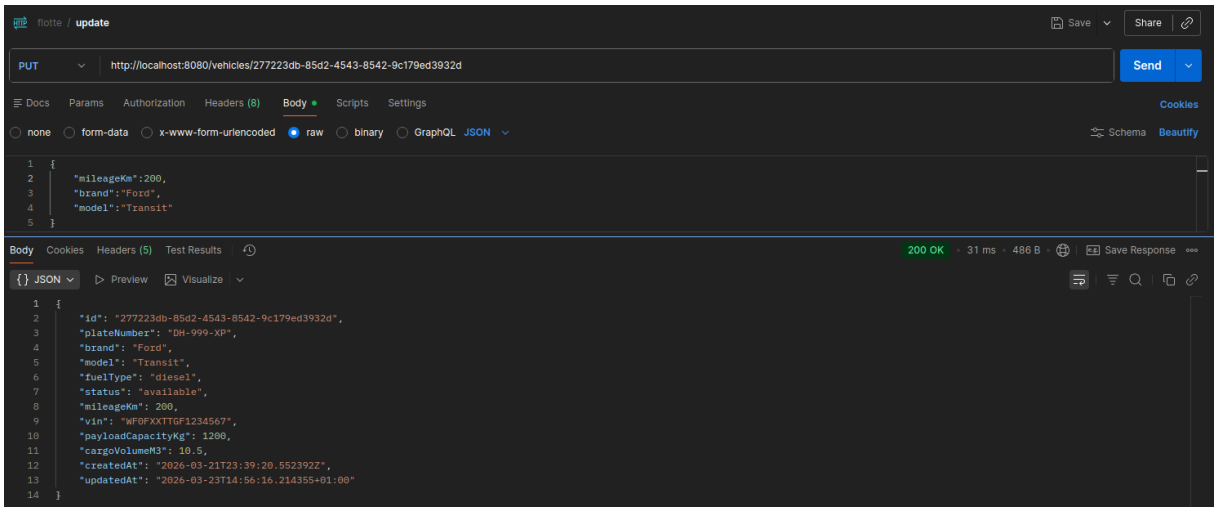

FIGURE 5 – PUT /vehicles/ :id — Mise à jour complète d'un véhicule (200 OK)

PATCH /vehicles/ :id/status — Mise à jour du statut

Une route dédiée permet de modifier uniquement le statut d'un véhicule (ex. `available`, `in_maintenance`, `on_delivery`). Cette approche évite d'envoyer l'intégralité du corps pour un changement ponctuel. Le service répond avec un **200 OK** et retourne le véhicule avec son nouveau statut.

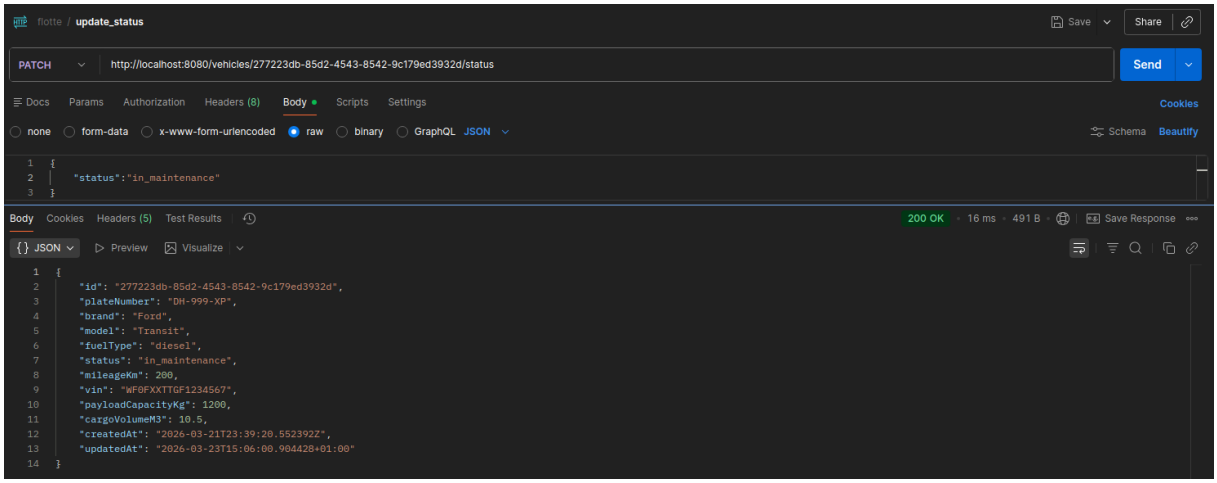

FIGURE 6 – PATCH /vehicles/ :id/status — Mise à jour du statut (200 OK)

DELETE /vehicles/ :id — Suppression logique

La suppression d'un véhicule est implémentée en **soft delete** : le véhicule n'est pas physiquement retiré de la base de données mais marqué comme supprimé. Le service retourne un **204 No Content**, et le véhicule disparaît des résultats des requêtes `GET`. La base de données (visible via pgAdmin) conserve bien les deux entrées, confirmant ce comportement.

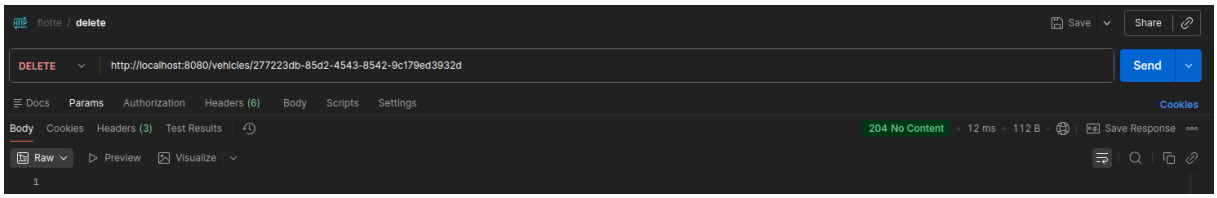

FIGURE 7 – DELETE /vehicles/ :id — Suppression logique (204 No Content)

3.2. Gestion des assignations

POST /vehicles/ :id/assignments — Ajout d'une assignation

L'assignation d'un conducteur à un véhicule s'effectue via une requête `POST` sur `/vehicles/:id/assi`
Le corps contient le `driverId`, des `notes` optionnelles et le `createdBy`. Le service répond avec un **201 Created** et retourne l'assignation créée avec sa date de début.

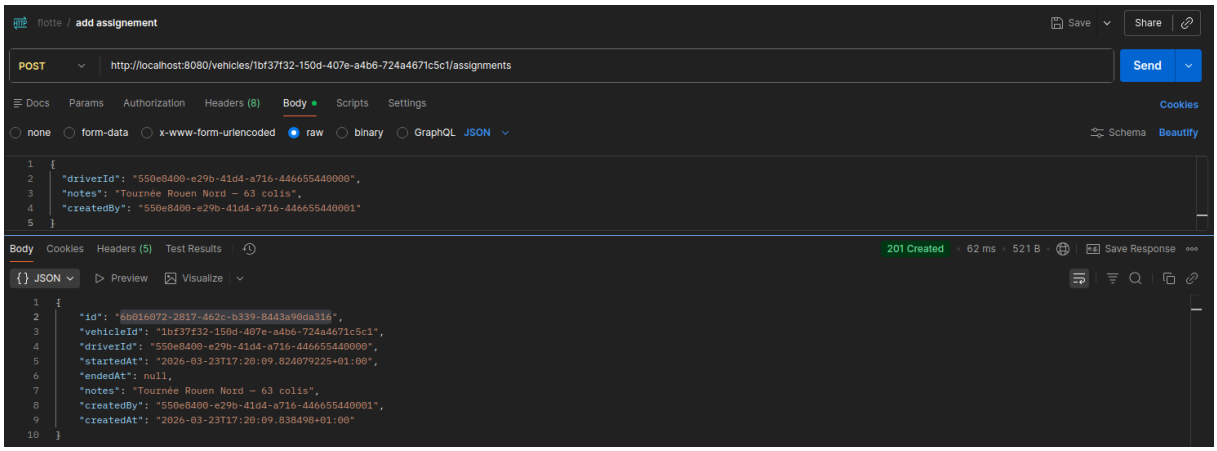

FIGURE 8 – POST /vehicles/ :id/assignments — Création d'une assignation (201 Created)

GET /vehicles/ :id/assignments — Liste des assignations

La récupération des assignations d'un véhicule retourne l'historique complet des affectations de conducteurs. Le champ `endedAt` à `null` indique une assignation en cours. Le service répond avec un **200 OK**.

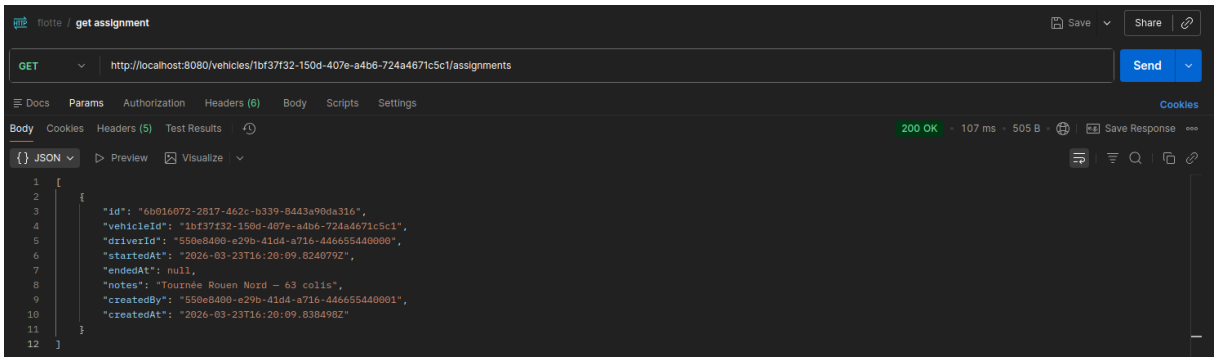

FIGURE 9 – GET /vehicles/ :id/assignments — Liste des assignations (200 OK)

3.3. Bilan des tests

Requête	Endpoint	Code attendu	Résultat
POST	/vehicles	201 Created	**OK**
GET	/vehicles	200 OK	**OK**
GET	/vehicles/ :id	200 OK	**OK**
PUT	/vehicles/ :id	200 OK	**OK**
PATCH	/vehicles/ :id/status	200 OK	**OK**
DELETE	/vehicles/ :id	204 No Content	**OK**
POST	/vehicles/ :id/assignments	201 Created	**OK**
GET	/vehicles/ :id/assignments	200 OK	**OK**

L'ensemble des 8 endpoints testés retournent les codes HTTP attendus. Le soft delete est correctement implémenté et le filtrage des véhicules supprimés fonctionne comme prévu.

4. Architecture événementielle avec Apache Kafka

Objectif : Garantir que chaque action critique sur le parc automobile (création, mise à jour de statut, suppression) est diffusée en temps réel au reste du système via un bus d'événements.

Le microservice véhicules fait office de **Producer** pour le topic `vehicle-events`. Cette approche événementielle permet un couplage faible : les futurs services (Alertes, Maintenance, Localisation) pourront réagir instantanément aux changements sans que le service véhicule ne connaisse leur existence.

4.1. Événements produits

Trois types d'événements sont actuellement émis par le service :

— `VEHICLE_CREATED` : Émis lors de l'ajout d'un nouveau véhicule.

— `VEHICLE_STATUS_UPDATED` : Émis lors d'un changement de statut ou de l'affectation d'un chauffeur.

— `VEHICLE_DELETED` : Émis lors d'une suppression logique (Soft Delete).

4.2. Consommation et validation des messages

Pour valider l'envoi correct des messages sur le bus, nous utilisons l'outil `kafka-console-consumer` intégré au conteneur Kafka. La commande suivante permet de visualiser en temps réel le flux des événements métier :

```
docker exec -it flotte-kafka /opt/kafka/bin/kafka-console-consumer.sh \
  --topic vehicle-events \
  --bootstrap-server localhost:9092 \
  --from-beginning
```

Listing 4 – Commande de visualisation des événements Kafka

Chaque message est sérialisé en **JSON** et contient l'identifiant du véhicule, le type d'événement, ainsi qu'un timestamp. Cette traçabilité est essentielle pour le débogage et l'audit du système.

5. Résolveurs GraphQL

Objectif : Exposer les données du service Véhicules via une couche **GraphQL** portée par l'API Gateway, en séparant clairement les résolveurs fonctionnels des résolveurs non encore branchés aux microservices.

L'API Gateway agrège les résolveurs de l'ensemble des microservices en un schéma GraphQL unifié. Pour la semaine 3, seuls les résolveurs liés au service Véhicules sont pleinement opérationnels. Les autres (conducteurs, alertes, localisation, maintenance) sont provisoirement remplacés par des **stubs** qui retournent une erreur explicite, évitant tout comportement silencieux.

5.1. Architecture des résolveurs

Les résolveurs sont organisés en trois fichiers distincts :

- `vehicleResolvers.ts` : résolveurs actifs, branchés sur le service Véhicules.
- `stubResolvers.ts` : résolveurs non encore implémentés, retournant une erreur `NOT_IMPLEMENTED`.
- `index.ts` : point d'entrée fusionnant les deux via spread operator.

Le fichier `index.ts` assemble les résolveurs de façon non destructive : les clés de `vehicleResolvers` écrasent celles des stubs pour les champs déjà implémentés, ce qui permet d'enrichir progressivement le schéma semaine après semaine sans modifier la structure d'agrégation.

5.2. Résolveurs Véhicules

Queries

Trois queries sont exposées et délèguent directement au contexte GraphQL (`ctx.vehicle`) :

- `vehicles` : liste des véhicules avec filtrage optionnel par statut (`status`, `limit`, `offset` avec valeurs par défaut de 20 et 0).
- `vehicle` : récupération d'un véhicule par son UUID.
- `vehicleAssignments` : récupération de l'historique des assignations d'un véhicule via son `vehicle_id`.

Mutations

Six mutations sont implémentées :

- `createVehicle` : création d'un véhicule avec ses champs obligatoires (`plate_number`, `brand`, `model`, `fuel_type`, `mileage_km`, `payload_capacity_kg`, `cargo_volume_m3`) et optionnels (`vin`, `color`).
- `updateVehicle` : mise à jour complète des champs modifiables d'un véhicule (`brand`, `model`, `mileage_km`, `vin`, `color`).
- `updateVehicleStatus` : mise à jour ciblée du seul statut d'un véhicule.
- `deleteVehicle` : suppression logique d'un véhicule (soft delete), retourne un booléen confirmant l'opération.
- `assignVehicle` : assignation d'un conducteur à un véhicule via leurs UUIDs.
- `unassignVehicle` : désassignation du conducteur actuel d'un véhicule.

Résolveurs de types

Le type `Vehicle` définit des résolveurs de champs pour les relations non encore implémentées (`current_location`, `current_assignment`, `maintenance_records`), retournant respectivement `null` ou un tableau vide. Cette approche permet au schéma de rester valide sans bloquer les requêtes côté client.

Le type `Assignment` résout ses deux relations :

- `vehicle` : récupère le véhicule lié via `ctx.vehicle.getVehicle()` en validant la présence du `vehicle_id`.
- `driver` : récupère le conducteur lié via `ctx.driver.getDriver()` en validant la présence du `driver_id`.

Dans les deux cas, une `GraphQLError` avec le code `NOT_FOUND` ou `INVALID_UPSTREAM` est levée si la donnée est absente, garantissant des messages d'erreur explicites plutôt qu'un `null` silencieux.

5.3. Cohérence REST / GraphQL

L'ensemble des endpoints REST du service Véhicules dispose d'un équivalent GraphQL, assurant une parité complète entre les deux interfaces. Trois opérations (`updateVehicle`, `deleteVehicle`, `vehicleAssignments`) ont été ajoutées au schéma initial pour combler les écarts identifiés.

Méthode	Endpoint REST	Équivalent GraphQL	Statut
POST	/vehicles	mutation `createVehicle`	**OK**
GET	/vehicles	query `vehicles`	**OK**
GET	/vehicles/ :id	query `vehicle`	**OK**
PUT	/vehicles/ :id	mutation `updateVehicle`	**OK**
PATCH	/vehicles/ :id/status	mutation `updateVehicleStatus`	**OK**
DELETE	/vehicles/ :id	mutation `deleteVehicle`	**OK**
POST	/vehicles/ :id/assignments	mutation `assignVehicle`	**OK**
GET	/vehicles/ :id/assignments	query `vehicleAssignments`	**OK**
DELETE	/vehicles/ :id/assignments/current	mutation `unassignVehicle`	**OK**

5.4. Gestion des stubs

Les résolveurs non encore branchés sont centralisés dans `stubResolvers.ts`. Deux comportements sont distingués :

— **Queries et Mutations** : lèvent une `GraphQLError` avec le code `NOT_IMPLEMENTED` et un message indiquant le champ concerné.

— **Subscriptions** : lèvent une erreur `SUBSCRIPTIONS_DISABLED`, signalant que le transport WebSocket (`graphql-ws`) n'est pas encore configuré sur le gateway.

Les champs concernés par les stubs couvrent les services à venir : `drivers`, `alerts`, `vehicleLocation`, `locationHistory` et `maintenanceRecords` pour les queries ; `createDriver`, `acknowledgeAlert`, `createMaintenanceRecord` et leurs variantes pour les mutations.

Cette approche **fail-fast** garantit qu'aucun appel à un service non implémenté ne passe silencieusement, facilitant le débogage lors des semaines suivantes.

6. Sécurisation avec Keycloak et RBAC

> **Objectif :** Mettre en place une authentification centralisée et un contrôle d'accès basé sur les rôles (RBAC) pour protéger les ressources du microservice véhicules.

La sécurité a été intégrée cette semaine via **Keycloak**, agissant comme fournisseur d'identité (IdP) et serveur d'autorisation. Le microservice véhicule agit en tant que **Resource Server** OAuth2.

6.1. Configuration de Keycloak

Nous utilisons un royaume (*realm*) nommé `gestion-flotte`. Les éléments suivants ont été configurés :

- **Rôles** : `admin` (gestion totale) et `technician` (gestion des statuts).
- **Import automatique** : Le royaume est exporté dans `realm-export.json` et importé automatiquement au démarrage du conteneur via la commande `-import-realm`.

6.2. Intégration Spring Security

Le microservice utilise `spring-boot-starter-oauth2-resource-server` pour valider les jetons JWT émis par Keycloak. Une configuration spécifique a été mise en place pour faire correspondre les rôles Keycloak aux autorités Spring Security.

6.2.1. Mapping des rôles

Par défaut, Spring Security ne décode pas les rôles spécifiques à Keycloak (stockés dans le champ `realm_access.roles` du jeton). Nous avons implémenté un `JwtAuthenticationConverter` personnalisé pour convertir ces rôles en `GrantedAuthority` avec le préfixe `ROLE_`.

```java
@Bean
public Converter<Jwt, AbstractAuthenticationToken> jwtAuthenticationConverter() {
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(jwt -> {
        Map<String, Object> realmAccess = jwt.getClaim("realm_access");
        Collection<String> roles = (Collection<String>) realmAccess.get("roles");
        return roles.stream()
            .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
            .collect(Collectors.toList());
    });
    return converter;
}
```

Listing 5 – Convertisseur de rôles JWT personnalisé

6.3. Contrôle d'accès (RBAC)

La sécurité est appliquée au niveau des méthodes du contrôleur grâce à l'annotation `@PreAuthorize`. Cela permet une gestion fine des permissions :

- **Lecture (GET)** : Accessible à tout utilisateur authentifié.
- **Création et suppression** : Réservé au rôle `admin`.
- **Mise à jour du statut** : Accessible aux rôles `admin` et `technician`.

```
@DeleteMapping("/{id}")
@PreAuthorize("hasRole('admin')")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteVehicle(@PathVariable UUID id) {
    service.deleteVehicle(id);
}
```

Listing 6 – Exemple de protection RBAC sur un endpoint

7. Instrumentation et Observabilité Distribuée

> **Objectif :** Garantir une visibilité complète sur l'état de santé et les performances du microservice via les trois piliers de l'observabilité (Traces, Métriques, Logs) en utilisant l'instrumentation automatique.

Pour cette troisième semaine, nous avons migré vers une instrumentation basée sur l'**Agent Java OpenTelemetry**. Contrairement à l'instrumentation manuelle, l'agent s'attache à la JVM au démarrage et intercepte automatiquement les appels système, les requêtes HTTP (Tomcat), les requêtes SQL (PostgreSQL) et les flux de logs (Logback).

7.1. Collecte des données via l'Agent Java

L'agent injecte du bytecode pour capturer la télémétrie sans modification intrusive du code source. Les données sont ensuite exportées via le protocole **OTLP** vers le collecteur centralisé :

- **Traces (Jaeger)** : L'agent capture l'intégralité du cycle de vie des requêtes. On peut ainsi visualiser précisément le temps passé dans les contrôleurs Spring, mais aussi les requêtes SQL générées par Hibernate.
- **Logs (Loki)** : Grâce à l'instrumentation de Logback, chaque ligne de log est capturée et enrichie d'un `trace_id`, permettant une corrélation immédiate entre un message d'erreur et une trace spécifique.
- **Métriques (Prometheus)** : L'agent expose nativement des métriques critiques comme l'utilisation CPU, la mémoire JVM (Heap/Non-Heap) et les statistiques du pool de connexions HikariCP.

7.2. Configuration de l'exportation (Docker Compose)

La configuration est pilotée par des variables d'environnement dans le fichier `compose.yaml`, assurant la liaison avec le collecteur sur le port `4318` (HTTP/Protobuf).

```
# Configuration pour l'Agent Java OpenTelemetry
- OTEL_SERVICE_NAME=vehicle-service
- OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector:4318
- OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf
- OTEL_LOGS_EXPORTER=otlp
- OTEL_TRACES_EXPORTER=otlp
```

Listing 7 – Variables d'environnement pour l'Agent Java

7.3. Visualisation et Diagnostic dans Grafana

L'exploitation des données se fait via l'interface **Explore** de Grafana. Voici les requêtes types utilisées pour valider le bon fonctionnement du service :

- **Loki (Logs)** : `{job="vehicle-service"}` ou `{service_name="vehicle-service"}` pour filtrer l'ensemble des logs applicatifs.

— **Prometheus (Métriques)** : `jvm_memory_used_bytes` pour suivre l'empreinte mémoire ou `http_server_requests_seconds_count` pour le volume de trafic.

— **Jaeger (Traces)** : Sélection du service `vehicle-service` pour analyser les latences et les dépendances vers la base de données.

8. Tests automatisés et couverture de code

> **Objectif :** Garantir la robustesse et la fiabilité du service véhicules via une suite de tests automatisés (unitaires et intégration) avec un objectif de couverture minimale de 80%.

La qualité logicielle est un pilier fondamental du projet. Pour assurer la non-régression et valider les règles métier (comme l'interdiction d'assigner un véhicule en maintenance), nous avons mis en place une stratégie de tests rigoureuse.

8.1. Stratégie de tests

L'approche adoptée combine deux niveaux de tests complémentaires :

— **Tests Unitaires** : Basés sur **JUnit 5** et **Mockito**. Ils isolent la logique métier de la couche `Service` et des `Controllers` en simulant les dépendances (Repositories, Kafka).

— **Tests d'Intégration** : Utilisent **MockMvc** et une base de données **H2** en mémoire pour valider le comportement réel de l'API, incluant la validation des données, les transactions JPA et le cycle de vie complet des requêtes.

8.2. Mesure de la couverture avec JaCoCo

Pour piloter cette démarche, nous utilisons **JaCoCo**. L'outil est intégré au pipeline Maven et génère un rapport à chaque exécution de la commande `mvn test`.

vehicle-service

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.flotte.vehicle.services		89 %		62 %	10	24	0	104	7	20	0	1
com.flotte.vehicle.models		88 %		n/a	5	45	5	52	5	45	0	2
com.flotte.vehicle.events.producers		37 %		n/a	1	3	3	7	1	3	0	1
com.flotte.vehicle		37 %		n/a	1	2	2	3	1	2	0	1
com.flotte.vehicle.dto		100 %		n/a	0	7	0	15	0	7	0	6
com.flotte.vehicle.events		100 %		n/a	0	6	0	6	0	6	0	1
com.flotte.vehicle.controller		100 %		n/a	0	10	0	13	0	10	0	1
com.flotte.vehicle.models.enums		100 %		n/a	0	2	0	10	0	2	0	2
Total	83 of 951	91 %	3 of 8	62 %	17	99	10	210	14	95	0	15

FIGURE 10 – Rapport de couverture JaCoCo — Service Véhicules

8.3. Analyse des résultats

Comme illustré par la capture ci-dessus, les résultats obtenus sont excellents et dépassent largement nos engagements :

— **Couverture globale (Instructions) : 91%** (Objectif initial : 80%).

— **Couverture des contrôleurs et DTOs : 100%** (Validation complète des endpoints).

— **Couverture de la couche Service : 89%**.

Le score de **62% sur les branches** s'explique par la présence de nombreux contrôles de garde et validations d'erreurs (cas limites) dont certains chemins d'exception n'ont pas encore été sollicités. Néanmoins, l'intégralité du "chemin nominal" et des règles métier critiques est couverte et validée.

9. Défis techniques et résolutions

Au cours du développement et de la mise en place de l'intégration continue (CI/CD), plusieurs obstacles techniques majeurs ont été rencontrés et résolus.

9.1. Incompatibilité de version Keycloak et initialisation

L'export du royaume (`realm-export.json`) utilisé pour l'automatisation contenait des configurations (notamment le champ `bruteForceStrategy`) spécifiques aux versions récentes de Keycloak (v26+). L'utilisation d'une image Docker plus ancienne (v24) provoquait l'échec critique du démarrage du serveur. De plus, l'initialisation automatique du compte administrateur via les anciennes variables d'environnement était ignorée dans certains contextes de déploiement.

Solutions :

— Mise à jour de l'infrastructure vers l'image `keycloak:latest`.

— Nettoyage chirurgical du fichier d'export pour retirer les champs obsolètes.

— Utilisation des nouvelles variables de "bootstrap" (`KC_BOOTSTRAP_ADMIN_*`) et configuration des en-têtes de proxy (`xforwarded`) pour garantir l'accès immédiat à l'interface d'administration sans configuration manuelle.

9.2. Gestion des dépendances Kafka

Un problème de gestion de version dans le `pom.xml` empêchait la compilation sur les agents de build CI/CD, car la dépendance `spring-boot-starter-kafka` ne récupérait pas de version par défaut. **Solution :** Alignement sur la dépendance `spring-kafka` (groupe `org.springframework.kafka`), permettant une gestion robuste des versions via le BOM (Bill of Materials) de Spring Boot 3.4.0.

9.3. Isolation de l'environnement de test

L'intégration de la sécurité OAuth2 et de l'observabilité a complexifié l'exécution des tests automatisés en environnement isolé :

— **Sécurité** : Les tests tentaient de valider les jetons auprès d'un serveur Keycloak réel. Nous avons résolu cela en **mockant le JwtDecoder** et en utilisant l'annotation `@WithMockUser` pour simuler les rôles RBAC (`admin`, `technician`).

— **Logging** : L'instrumentation OpenTelemetry pour les logs provoquait des erreurs `NoClassDefFoundError` en mode test.

— **Solution** : Création d'un profil de logging spécifique (`logback-test.xml`) excluant l'appender OpenTelemetry durant la phase de test, garantissant ainsi un build fluide et rapide dans le pipeline GitHub Actions.